

Matmul, but with CUDA

MGP 2025 Spring

Due: 30 APR (WED) 23:59

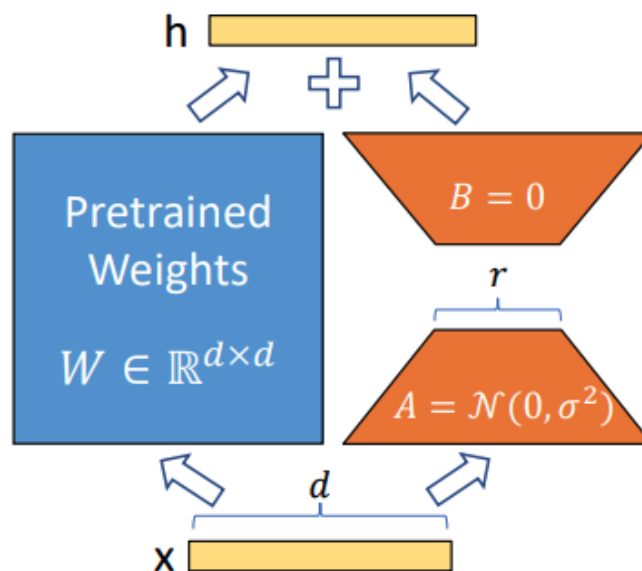
Hello, MGP class!

You're now becoming familiar with matrix multiplication, and in this assignment, we're taking it a step further by implementing matrix multiplication (matmul) using CUDA C++.

Additionally, we'll incorporate this Triton-based matmul into one of the most powerful and widely used deep learning techniques—PEFT, specifically LoRA.

Don't worry if you haven't heard of LoRA before.

You don't need to know all the details about what it is or why it's used here. Just keep in mind that LoRA relies heavily on matrix multiplication, and that's what you'll be implementing in CUDA C++.



This is LoRA. Don't worry. All of the things are matmul.

The numpy implementation of LoRA is:

```
import numpy as np

def lora(x, W, A, B, alpha):
    """
    NumPy implementation of LoRA-augmented linear layer

    Args:
        x: [B, in_dim] - input matrix
        W: [out_dim, in_dim] - original weight
        A: [r, in_dim] - LoRA down projection
        B: [out_dim, r] - LoRA up projection
        alpha: scaling factor

    Returns:
        y: [B, out_dim] - output after applying LoRA
    """
    r = A.shape[0]
    scale = alpha / r
    out_linear = x @ W.T # [B, out_dim]
    out_lora = x @ A.T @ B.T * scale # [B, out_dim]
    return out_linear + out_lora
```

Alright, let's dive into CUDA and get started!

[Overview]

Implement the LoRA in CUDA C++ in `lora.h`.

[Code - 10 pts]

Your submission will be graded out of a total of 10 points based on its performance and ranking relative to other submissions:

- 10 pts: Awarded if your program ranks in the **top 10%**.
- 9 pts: Awarded if your program ranks in the **top 30%**.
- 8 pts: Awarded if your program ranks in the **top 50%**.
- 7 pts: Awarded if your program functions correctly.
 - Your maximum absolute error must be less than **0.01**.

Bonus points are awarded if your program outperforms and is not a late submission.

- 2 pt, if you are the best.

Late submissions that produce a functioning solution will receive 4 points for this assignment.

Be sure to optimize your solution to achieve the highest ranking possible.

[Report - 10 pts]

You have to submit a "good" report about your homework, including

1. How did you implemented your parallel algorithm
2. Reasons why your program is performing at its best and proving whether performance margin is there or not
3. Evaluation including tables or graphs
4. Something else

[Note]

Your code's speed is measured based on the maximum of 5 executions on the grading server. I won't regrade due to "It worked on my machine" excuses, so I encourage you to perform the assignment more generously than the criteria. As many students tend to rush near the deadline, your code may appear slower in the local environment. Therefore, I

recommend completing the assignment **as early as possible** when the server is less congested.

[Rules]

1. Put your HW3 directory under <your_home>/mgp/hw3
 - a. All your make commands, especially 'make run' should work.
2. **Do not change any** of the output(i.e. 'std::cout') statements. We will grade based on the program outputs. Any change on the outputs would probably cause an error, leading to 0 points for your HW assignment.
3. Don't touch anything in the directory after the deadline. If any of the timestamps are later than the deadline, we will treat it as 'late submission'.
4. You can't use OpenMP for this assignment.
5. Please retain adequate permission on your files to avoid plagiarism.
6. No abusing. Please use your common sense.
7. Zero GPT policy. No mercy.
8. **PLEASE** 本當に 제발 *read thoroughly this document.*